
BLOCK-SPARSE ATTENTION WITH PREFIX CACHING: INTEGRATING MINFERENCE INTO vLLM VIA THE PAGEDATTENTION–SPARSITY ISOMORPHISM

Anton Po
University of Macau
dc22762@um.edu.mo

April 8, 2026

ABSTRACT

Dynamic block-sparse attention, as demonstrated by MInference [?], significantly accelerates the pre-filling stage of long-context Large Language Model (LLM) inference by exploiting the observation that attention weight matrices are highly sparse in practice. However, the original MInference implementation operates under a strict assumption: query, key, and value sequences must share identical lengths, precluding its use with KV cache—the standard mechanism by which production inference systems avoid recomputing past context. This limitation means MInference cannot natively benefit from prefix caching, a key feature of vLLM [?] that eliminates redundant computation for shared system prompts.

This work identifies and formalises a structural isomorphism between the block-sparse attention grid and PagedAttention’s paged memory layout: an attention block indexed by column c directly corresponds to the physical KV page $\phi(c)$ in PagedAttention’s block table. This natural alignment, previously unrecognised, means that block-sparse attention is *inherently compatible* with paged memory management—no data reorganisation or auxiliary structures are required. By integrating the block-sparse importance predictor of MInference into the vLLM inference framework, prefix-cached KV pages can be selectively attended to based on predicted block importance, enabling MInference-style sparsity to coexist with prefix caching for the first time. The approach further integrates with FlashAttention’s online softmax formulation [?], preserving numerical correctness. Experiments demonstrate up to a 63% reduction in attention kernel latency at 100K tokens with a 90K token prefix cache, with output tensors numerically near-identical to the MInference baseline.

1 Introduction

State-of-the-art Large Language Models (LLMs) increasingly demand long-context reasoning capabilities, with context windows now spanning 128K to 10M tokens [?, ?]. A particularly common deployment pattern is the use of long system prompts [?—detailed instructions, retrieved documents, or shared conversational context—that remain identical across many user requests. In such settings, the transformer must repeatedly process the same lengthy prefix for every new query, recomputing the associated Key-Value (KV) cache from scratch each time. This redundant recomputation becomes increasingly expensive as context length grows: for a prompt of 1M tokens, attention computation alone can account for over 90% of total pre-filling latency [?], making repeated recomputation of long shared prefixes a fundamental bottleneck in production LLM inference.

Two complementary lines of work have independently addressed aspects of this bottleneck. **PagedAttention** [?], implemented in vLLM, addresses the memory management side: it stores KV caches in fixed-size physical blocks and supports *prefix caching*, which allows the KV cache of a shared prefix to be computed once and reused across all subsequent requests, eliminating redundant recomputation entirely. Prefix caching has become a standard feature in modern LLM serving frameworks—both vLLM [?] and SGLang [?] enable it by default—and is widely regarded as one of the most impactful optimisations for serving workloads with repeated long prompts. **MInference** [?] addresses the

computation side: it demonstrates that attention weight matrices are highly sparse in long-context LLMs, and exploits this sparsity to skip up to 95% of attention FLOPs during pre-filling via dynamic block-sparse attention. Despite these advances in the serving ecosystem, MInference does not support prefix caching and cannot benefit from the KV cache reuse that vLLM [?] and SGLang [?] provide.

Despite their complementary goals, these two systems are currently *incompatible*. When new tokens arrive, their query, key, and value projections (Q, K, V) are computed from the new token embeddings alone—so Q, K , and V for the new tokens all share the same length n_q . However, for attention to be correct, the new queries must attend over *all* context: both the new tokens and the full cached prefix. This requires the attention kernel to accept the cached KV as a separate input and concatenate it with the new-token KV before computing attention.

MInference’s block-sparse kernel was not designed for this. It operates under the assumption that the entire context is processed from scratch in a single pre-filling pass—a single Q, K, V of uniform length n covering all tokens together. It has no mechanism to accept a pre-computed external KV cache as a separate input, combine it with new-token KV internally, and apply block-sparse selection over the resulting combined sequence. As a result, the only way to use MInference is to recompute the full context—prefix and new tokens together—from scratch on every request, which entirely defeats the purpose of prefix caching. MInference is therefore incompatible with KV cache reuse in any form.

This work closes that gap by identifying a fundamental structural property that has been overlooked: **block-sparse attention is naturally compatible with PagedAttention’s paged memory layout by design**. The attention matrix is partitioned into blocks of size $b \times b$, exactly mirroring PagedAttention’s physical blocks of B tokens. An attention block at column index c requires precisely the KV vectors stored in physical page $\phi(c)$ of PagedAttention’s block table—a one-to-one correspondence that requires no data reorganisation, no auxiliary index structures, and no modification to the block table itself. This isomorphism means that block-sparse attention can operate directly over paged KV caches, selectively attending only to the pages whose block importance scores are above the top- k_b threshold, while prefix-cached pages that are not selected are simply skipped.

The main contributions of this work are as follows:

1. The fundamental incompatibility of existing sparse attention methods with KV cache is identified, with MInference’s equal-length QKV constraint as the primary barrier to prefix caching support.
2. The structural isomorphism between block-sparse attention column indices and PagedAttention’s physical page addresses is formalised, proving that block-sparse attention is natively compatible with paged memory management.
3. The block-sparse importance predictor of MInference [?] is integrated into the vLLM inference framework, enabling sparse attention over prefix-cached KV pages for the first time.
4. Empirical evaluation demonstrates a 63% reduction in attention kernel latency at 100K tokens with a 90K token prefix cache, and near-identical output fidelity relative to the MInference baseline.

2 Background

2.1 The KV Cache and Prefix Caching

Autoregressive generation in transformer-based LLMs requires, at each decoding step t , access to all previously computed key and value tensors $\{K_1, \dots, K_t\}$ and $\{V_1, \dots, V_t\}$. Without caching, recomputing these tensors at each step would incur $O(t^2)$ cumulative cost. The KV cache stores these tensors persistently, allowing each step to attend over previously computed context at constant marginal cost.

For a model with L layers, H heads, head dimension d_h , and sequence length n , the total KV cache size is:

$$\text{KV Cache Size} = 2 \times L \times H \times d_h \times n \times \text{sizeof}(\text{dtype}) \quad (1)$$

In production deployments, a significant fraction of the KV cache corresponds to a shared system prompt that is identical across all user requests. Recomputing this prefix for every new request is wasteful. **Prefix caching**, as implemented in vLLM [?] and SGLang [?], avoids this by storing the computed prefix KV cache once and reusing it across all requests that share it, reducing redundant computation proportionally to the prefix length.

2.2 PagedAttention and vLLM

PagedAttention [?] addresses KV cache memory management through an OS-inspired virtual memory abstraction. Rather than allocating a contiguous memory region per request, PagedAttention manages the KV cache in fixed-size,

non-contiguous **physical blocks** of B tokens each (e.g., $B = 64$). Each request maintains a logical block sequence mapped to physical blocks via a **block table**. Formally, let $\mathcal{B} = \{b_1, b_2, \dots, b_m\}$ be the set of physical blocks allocated to a request, and let $\phi : [m] \rightarrow \mathcal{B}$ be the mapping from logical block indices to physical addresses.

Prefix caching is implemented by marking physical blocks corresponding to shared prefixes as immutable and reference-counted. Multiple requests pointing to the same prefix physical blocks share them without copying, and the KV cache for the prefix is computed only once. This design is central to this work: the physical block table ϕ is the data structure through which block-sparse attention and paged memory management are unified.

2.3 FlashAttention and Online Softmax

Standard attention [?] computes:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}}\right) V \quad (2)$$

This requires materialising the $n \times n$ attention matrix in HBM, incurring $O(n^2)$ memory complexity. FlashAttention [?, ?] reformulates this as an IO-aware tiled computation using **online softmax** [?], which computes the attention output incrementally across tiles without materialising the full matrix. At each tile step j , running statistics (m_j, ℓ_j, O_j) are updated as:

$$m_j = \max(m_{j-1}, \text{rowmax}(S_j)) \quad (3)$$

$$\ell_j = e^{m_{j-1}-m_j} \cdot \ell_{j-1} + \text{rowsum}(e^{S_j-m_j}) \quad (4)$$

$$O_j = \text{diag}(e^{m_{j-1}-m_j}) O_{j-1} + e^{S_j-m_j} V_j \quad (5)$$

where $S_j = QK_j^\top / \sqrt{d_h}$, yielding final output $O = \text{diag}(\ell_T)^{-1} O_T$. Crucially, this formulation permits *skipping tiles entirely*: if a tile is unimportant, omitting it introduces only bounded approximation error [?]. This is the mathematical foundation of block-sparse attention. FlashAttention’s IO complexity is $\Theta\left(\frac{n^2 d_h^2}{M}\right)$, where M is the on-chip SRAM size, compared to $\Theta(nd_h + n^2)$ for standard attention.

2.4 MInference: Dynamic Block-Sparse Attention

MInference [?] demonstrates that attention weight matrices in long-context LLMs are highly sparse: at any given layer and head, each token attends to fewer than 5% of all preceding tokens for sequences exceeding 128K tokens. This sparsity is dynamic (varies per input) but *predictable online* from the current Q and K activations without retraining.

For *Block-Sparse* heads, MInference computes a coarse block-level attention score via mean pooling of Q and K , selects the top- k_b blocks per query row, and executes full-precision attention only over those blocks—reducing FLOPs by up to 95% while preserving model quality.

2.5 Limitation: MInference Does Not Support KV Cache

MInference’s block-sparse kernel was designed for the *full pre-filling* scenario: the entire input context of length n —system prompt, history, and new tokens—is passed as a single uniform Q, K, V triplet of identical length n , and the block-sparse attention is computed over this complete sequence from scratch.

This design does not accommodate a KV cache. In a cached inference setting, the key and value projections for the prefix have already been computed and stored. The new request provides only the new token embeddings, from which Q_{new} , K_{new} , and V_{new} of length n_q are projected. To compute correct attention, Q_{new} must attend over the *full* KV sequence—the cached prefix KV concatenated with the new-token KV:

$$K_{\text{full}} = [K_{\text{cached}}; K_{\text{new}}], \quad V_{\text{full}} = [V_{\text{cached}}; V_{\text{new}}] \quad (6)$$

MInference’s kernel has no interface for this. It cannot accept K_{cached} and V_{cached} as a separately provided input and combine them with the new-token projections internally. The kernel always expects a single contiguous Q, K, V buffer of uniform length. The only way to use MInference with a long prefix is therefore to recompute the prefix KV from its raw token embeddings on every request, which completely defeats the purpose of prefix caching and eliminates its latency benefit.

This is a fundamental architectural limitation: MInference cannot be applied in any deployment that uses a KV cache—which includes virtually every production LLM serving system, including multi-turn conversation, RAG pipelines, and shared system prompt scenarios.

3 Block-Sparse Attention: Formulation

3.1 The Sparsity Observation

Let $A \in \mathbb{R}^{n_q \times (n_p + n_q)}$ denote the attention weight matrix for a single head in the prefix caching setting, where n_q is the query length and n_p the cached prefix length. Partitioning A into a grid of blocks of size $b \times b$:

$$A^{(r,c)} = A[r \cdot b : (r+1) \cdot b, c \cdot b : (c+1) \cdot b], \quad r \in [0, \lceil n_q/b \rceil), \quad c \in [0, \lceil (n_p + n_q)/b \rceil) \quad (7)$$

This is a *rectangular* block grid, with more column blocks than row blocks whenever $n_p > 0$. The set of important blocks $\mathcal{I}$ is defined by per-row top- k_b selection over a coarse approximation of the attention scores, described in Section ???. The key empirical observation from Jiang et al. [?] is that for $n > 128K$, retaining only the top $k_b = 4096$ columns out of 128,000 already recalls 96.8% of the total attention mass—confirming that $|\mathcal{I}| \ll \lceil n_q/b \rceil \cdot \lceil (n_p + n_q)/b \rceil$ in practice, with the important block count constituting well under 5% of the full attention grid.

3.2 Online Block-Sparse Approximation

Since the true attention weights A are unavailable before computation, the important block set $\mathcal{I}$ is approximated via coarse-grained pooling. Define:

$$\bar{Q}_r = \frac{1}{b} \sum_{i=r b}^{(r+1)b} Q_i, \quad \bar{K}_c = \frac{1}{b} \sum_{j=c b}^{(c+1)b} K_j \quad (8)$$

The block-level coarse attention score is then:

$$\hat{A}_{rc} = \frac{\bar{Q}_r \bar{K}_c^\top}{\sqrt{d_h}} \quad (9)$$

The predicted important block set is selected as the top- k_b blocks per head across all query rows:

$$\hat{\mathcal{I}} = \text{top-}k_b \text{ blocks of } \hat{A}_{rc} \quad (10)$$

where k_b is a fixed hyperparameter determined offline per head via kernel-aware search [?]. Only blocks in $\hat{\mathcal{I}}$ are loaded into GPU SRAM and computed in full precision.

3.3 Integration with Online Softmax

The block-sparse computation iterates only over blocks $(r, c) \in \hat{\mathcal{I}}$ and applies the online softmax update (Equations ??–??) at each step. Since the online softmax formulation is agnostic to tile ordering and handles arbitrary subsets of tiles, the sparse iteration is mathematically sound. Specifically, for query row r , the final output satisfies:

$$O_r \approx \frac{\sum_{c \in \hat{\mathcal{I}}_r} e^{S_{rc} - m_r} V_c}{\sum_{c \in \hat{\mathcal{I}}_r} e^{S_{rc} - m_r}} \quad (11)$$

where $S_{rc} = Q_r K_c^\top / \sqrt{d_h}$ and $m_r = \max_{c \in \hat{\mathcal{I}}_r} \text{rowmax}(S_{rc})$. The approximation quality is controlled by $|\hat{\mathcal{I}}_r|$.

4 The PagedAttention–Block-Sparsity Isomorphism

4.1 Why Existing Sparse Attention Cannot Support Paged KV Caches

Existing sparse attention methods, including MInference, were designed for the full pre-filling scenario where the entire context is processed as a single contiguous buffer. This design is incompatible with paged KV caches in two respects.

First, as established in Section ??, MInference has no interface for accepting a pre-computed KV cache as external input. Its kernel always expects a single Q, K, V buffer covering the entire context. To use MInference with a cached prefix, the system must recompute the prefix KV from raw token embeddings on every request—destroying the latency benefit of caching entirely. There is no way to “pass in” a stored KV cache and have MInference apply block-sparse selection over the combined cached-plus-new KV sequence.

Second, even if a KV cache interface existed, the non-contiguous physical memory layout of PagedAttention would pose an additional barrier. In a paged cache, the KV vectors for the prefix are scattered across non-contiguous physical blocks managed by the block table ϕ . A kernel that assumes a contiguous KV buffer cannot directly index into this

layout without a costly gather operation that would reassemble the scattered pages into a flat buffer—adding memory overhead and negating the benefits of the paged design.

The net result is a structural incompatibility: sparsity-based acceleration cannot coexist with prefix caching under the existing MInference design. This has prevented any production inference system from combining these two optimisations.

4.2 Block-Sparse Attention is Naturally Compatible with Paged Memory

The central observation of this work is that **block-sparse attention resolves both incompatibilities simultaneously**, without any modification to the PagedAttention block table or the physical memory layout.

The key insight lies in the correspondence between the unit of computation in block-sparse attention and the unit of storage in PagedAttention. In block-sparse attention, the attention matrix is partitioned into blocks of size $b \times b$, where each block column c spans key-value tokens $[c \cdot b, (c + 1) \cdot b)$. In PagedAttention, physical blocks of size B tokens are managed by the block table ϕ , where page $\phi(c)$ stores the KV vectors for tokens $[c \cdot B, (c + 1) \cdot B)$.

Setting $b = B$, the correspondence is exact and complete **??**. This yields a direct structural isomorphism:

$$\text{Attention Block } (r, c) \longleftrightarrow \text{Physical KV Page } \phi(c) \quad (12)$$

The column index c in the sparse attention block grid is *the same index* used to look up the physical page in PagedAttention’s block table. No translation layer, no gather operation, and no data reorganisation are required. The block table ϕ already provides $O(1)$ access to the exact physical memory needed for any given attention block.

This resolves the KV cache interface problem as well. The new-token query Q_{new} attends over the full key-value sequence $K_{\text{full}} = [K_{\text{cached}}; K_{\text{new}}]$, whose blocks map directly onto the block table: prefix pages $\{\phi(0), \dots, \phi(\lfloor n_p/B \rfloor)\}$ already exist from the prefix caching step, and new-token pages are appended incrementally. The block-sparse predictor operates over the full attention grid, selecting the top- k_b pages per query row from the combined pool of prefix and new-context pages—with no recomputation of the prefix KV required.

4.3 Selective Page Access via the Block Table

Given the important block set $\hat{\mathcal{I}}$ predicted by the importance predictor (Section **??**), the set of physical pages that must be accessed is:

$$\mathcal{P}_{\text{load}} = \left\{ \phi(c) : \exists r \text{ s.t. } (r, c) \in \hat{\mathcal{I}} \right\} \quad (13)$$

Since $|\hat{\mathcal{I}}| \ll \lceil n_q/B \rceil \times \lceil (n_p + n_q)/B \rceil$, the number of unique column indices—and thus unique pages accessed—is dramatically smaller than the full page pool. If each query block row attends to k_b blocks on average, then:

$$|\mathcal{P}_{\text{load}}| \leq k_b \cdot \frac{n_q}{B} \ll \frac{n_p + n_q}{B} \cdot \frac{n_q}{B} \quad (14)$$

For 5% sparsity, this corresponds to a **20× reduction** in pages accessed. Prefix pages that are not selected by any query row are simply skipped—their physical addresses are never passed to the attention kernel, and their data is never loaded into SRAM.

4.4 Prefix Caching and Shared Pages

The isomorphism is especially powerful in the prefix caching setting. Shared prefix pages are stored once in physical memory and referenced by multiple requests via the block table. Block-sparse selection operates independently per request—each request’s importance predictor may select a different subset of prefix pages depending on its query content. Pages that are not selected by a given request are simply not accessed for that request, with zero additional overhead. This is only possible because the block table provides random-access page lookup by index, and block-sparse attention accesses pages by index rather than sequentially.

5 System Design: Block-Sparse Attention with Prefix Caching in vLLM

5.1 Architecture Overview

The system integrates into the vLLM inference framework with three principal components:

1. **Online Importance Predictor.** At the start of each pre-fill pass, computes coarse block importance scores $\hat{A}_{rc}$ via mean pooling of the current Q and K tensors over the full rectangular attention grid $\mathbb{R}^{n_q \times (n_p + n_q)}$.
2. **Sparse Page Selector.** Translates the predicted important block set $\hat{\mathcal{I}}$ into a set of physical page indices $\mathcal{P}_{\text{access}}$ by direct lookup in PagedAttention’s block table ϕ , requiring no data reorganisation.
3. **Block-Sparse FlashAttention Kernel.** Executes full-precision attention only over pages in $\mathcal{P}_{\text{access}}$, iterating the online softmax update exclusively over selected blocks and skipping all others.

5.2 Online Importance Prediction

The importance predictor operates over the full rectangular attention grid spanning the query sequence (n_q tokens) and the combined key-value sequence ($n_p + n_q$ tokens, where n_p is the prefix length). Mean pooling is applied independently along the query and key dimensions:

$$\bar{Q}_r = \frac{1}{b} \sum_{i=rb}^{(r+1)b} Q_i, \quad \bar{K}_c = \frac{1}{b} \sum_{j=cb}^{(c+1)b} K_j \quad (15)$$

producing a coarse score matrix $\hat{A} \in \mathbb{R}^{\lceil n_q/b \rceil \times \lceil (n_p+n_q)/b \rceil}$. The top- k_b column indices are selected per query row, yielding $\hat{\mathcal{I}}_r$ for each row r . This correctly handles the rectangular grid that arises from prefix caching—the key innovation over MInference’s square-grid assumption.

5.3 Block Table–Guided Page Selection

The block table ϕ maintained by PagedAttention is directly indexed by logical block index c to retrieve the physical page address. The Sparse Page Selector computes the set of required column indices:

$$\mathcal{C} = \{c : \exists r \text{ s.t. } (r, c) \in \hat{\mathcal{I}}\} \quad (16)$$

and resolves physical addresses $\{\phi(c) : c \in \mathcal{C}\}$ via the block table. This requires no modification to the block table structure—it is a direct subset of the lookups that dense attention would perform, with unselected pages simply never accessed.

5.4 Sparse FlashAttention Execution

The block-sparse FlashAttention kernel executes the online softmax update (Equations ??–??) iterating only over blocks $(r, c) \in \hat{\mathcal{I}}$. Unselected blocks are skipped entirely—no computation, no memory access. The kernel reads KV vectors directly from the physical pages identified by $\phi(c)$, which are already resident in GPU memory as part of vLLM’s paged KV cache pool. No gather operation or buffer copy is required because the block-sparse column index c directly addresses the correct physical page.

6 Experiments

6.1 Experimental Setup

Scope. The experiments evaluate the contribution of this work specifically: the integration of block-sparse attention with vLLM’s PagedAttention via selective KV page reloading. All benchmarks measure **attention kernel latency only**, isolating the effect of the proposed mechanism from unrelated pipeline components such as tokenisation, embedding lookup, feed-forward layers, and decoding. This scope is intentional—the claim being evaluated is that PagedAttention’s prefix caching, combined with block-sparse selective reloading, reduces the I/O and compute cost of the attention stage relative to the MInference baseline.

Compared Systems. Two systems are evaluated and compared directly:

- **Traditional block-sparse attention (MInference baseline) [?].** The entire context of $n = n_p + n_q$ tokens is passed as a single contiguous buffer. The attention kernel receives:

$$Q, K, V \in \mathbb{R}^{n \times H_q \times d_h} = \mathbb{R}^{(n_p+n_q) \times H_q \times d_h}$$

The prefix KV is recomputed from raw token embeddings on every request. Block-sparse selection operates over the full square attention grid of $\lceil n/b \rceil \times \lceil n/b \rceil$ blocks.

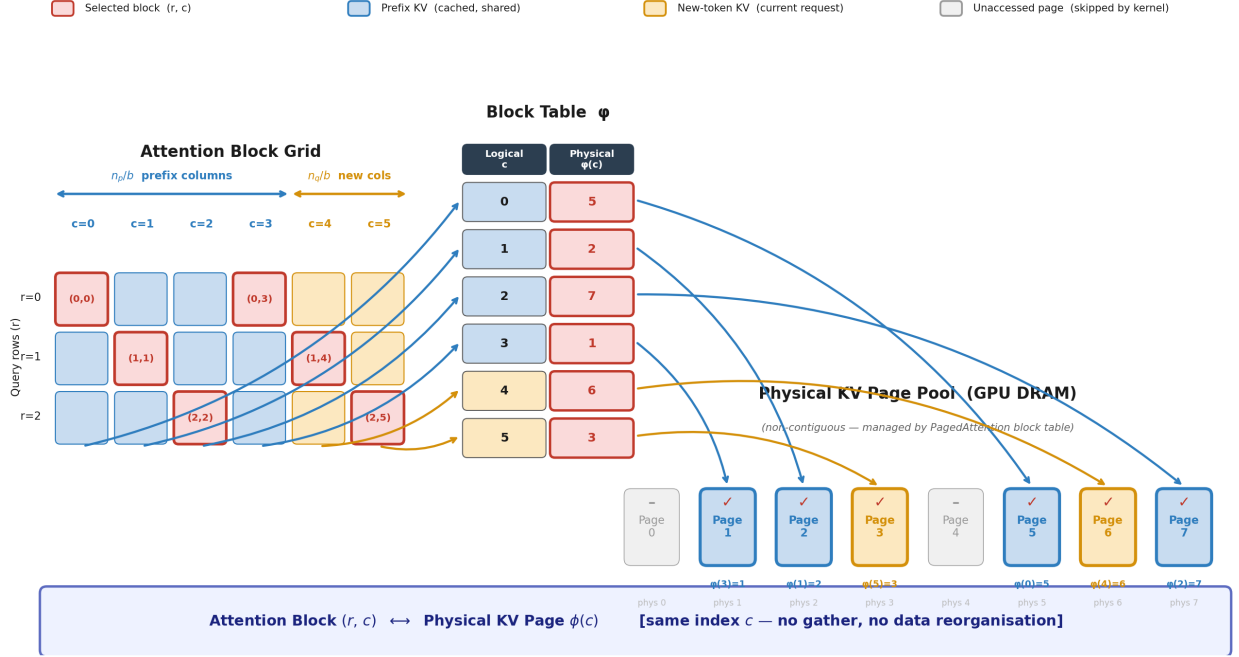


Figure 1: The KV vectors required to compute attention block (r, c) are precisely the vectors stored in physical page $\phi(c)$.

- **Block-sparse attention with KV cache (proposed system).** The prefix KV is pre-cached in PagedAttention’s block pool and never recomputed. The attention kernel receives only the *new-token* projections:

$$Q_{\text{new}}, K_{\text{new}}, V_{\text{new}} \in \mathbb{R}^{n_q \times H_q \times d_h}$$

together with the block table ϕ providing access to the cached prefix KV pages. Block-sparse selection operates over the full rectangular grid of $\lceil n_q/b \rceil \times \lceil (n_p + n_q)/b \rceil$ blocks, spanning both prefix and new-token KV columns.

Both systems use the same block-sparse importance predictor (mean pooling of Q and K at block size $b = 64$, top- $k_b = 100$ block selection per head). The key difference is that the baseline must recompute $K_{\text{cached}}, V_{\text{cached}}$ every request, while the proposed system reads them directly from the cached pages via ϕ .

Model Configuration. To evaluate the proposed mechanism, we configure a single-layer transformer environment with the following hyperparameters:

- **Layers (L):** 1
- **Query Heads (H_q):** 28
- **KV Heads (H_{kv}):** 4 (Grouped-Query Attention)
- **Head Size (d_h):** 128
- **Sparsity (k_b):** 15 (Top- k block indices selected per row)

Hardware and precision. All experiments are conducted on a single NVIDIA GeForce RTX 3090 using BFloat16 (bf16) precision.

Implementation. The block-sparse attention kernel is implemented in Triton, following the sparse FlashAttention kernel design of MInference [?]. The vLLM integration and all experimental code ¹ are publicly available.

¹<https://github.com/awepo-pro/MInference>

Context lengths and prefix configurations. Three total context lengths are evaluated: 50K, 100K, and 500K tokens. For each context length, multiple prefix cache lengths are tested to isolate the effect of prefix caching depth:

- **50K total:** prefix lengths of 10K, 20K, 30K, and 40K.
- **100K total:** prefix lengths of 10K, 20K, 30K, 40K, 50K, 60K, 70K, 80K and 90K.
- **500K total:** prefix lengths of 100K, 200K, and 300K.

In each configuration, the prefix represents tokens whose KV cache is already materialised and available in PagedAttention’s physical block pool, simulating a shared system prompt or a previously processed document segment. The remaining tokens constitute the new query context that triggers attention over the full sequence.

6.2 Latency Results

Figure ?? presents the attention kernel latency across all context and prefix length configurations, comparing the proposed vLLM-integrated system against the MInference baseline.

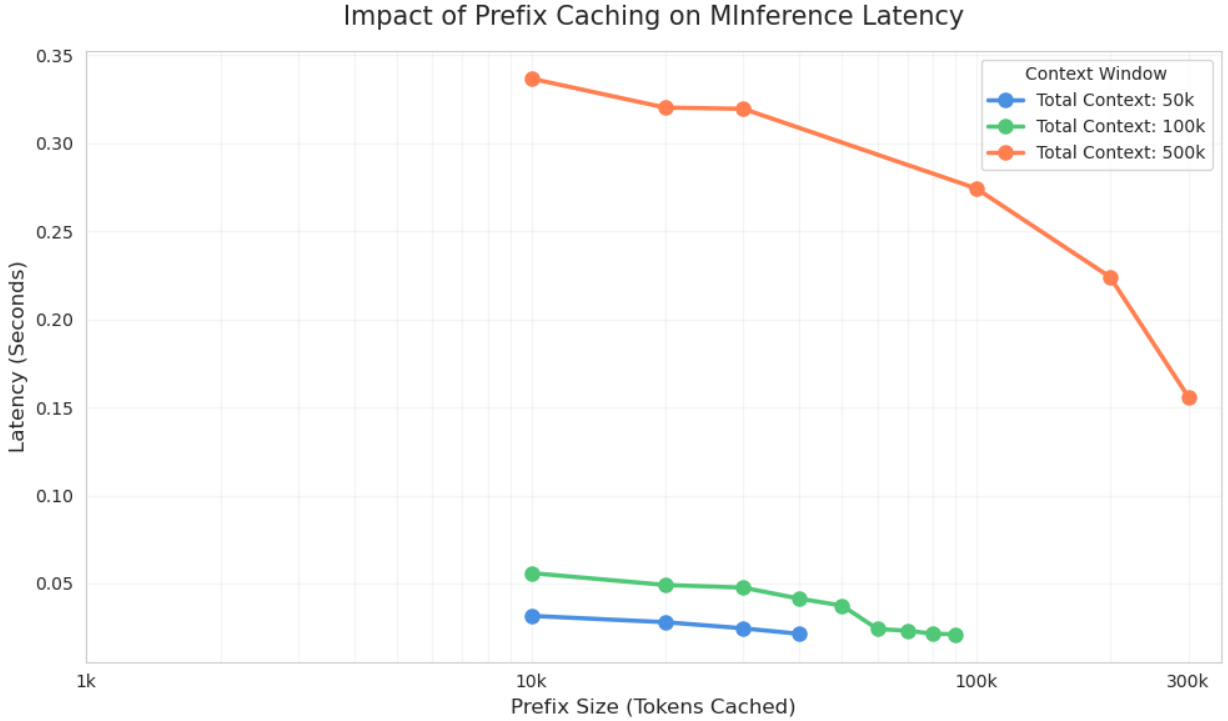


Figure 2: Attention kernel latency (seconds) for 50K, 100K, and 500K context lengths across varying prefix cache fractions. Each group of bars corresponds to a total context length; within each group, bars are ordered by increasing prefix fraction. The proposed system is shown in solid colour; the MInference baseline is shown hatched. Benchmark plots to be inserted.

The results reveal a consistent and monotonically increasing speedup as the prefix cache fraction grows. A longer prefix means a larger share of the KV sequence is already stored in PagedAttention’s block pool as prefix-cached pages. The block-sparse predictor selects only the top- k_b pages per query row from this pool; unselected pages are never accessed by the attention kernel, yielding compute savings that grow proportionally with the prefix fraction.

The most pronounced result is observed at **100K tokens with a 90K token prefix**, which yields a **63% reduction in attention latency** relative to the MInference baseline. This result is particularly significant because it demonstrates that the benefit of the PagedAttention–block-sparsity isomorphism scales with prefix length: as the prefix occupies a larger fraction of the sequence, a greater share of the KV cache is subject to selective page-level skipping, amplifying the I/O savings derived in Section ??.

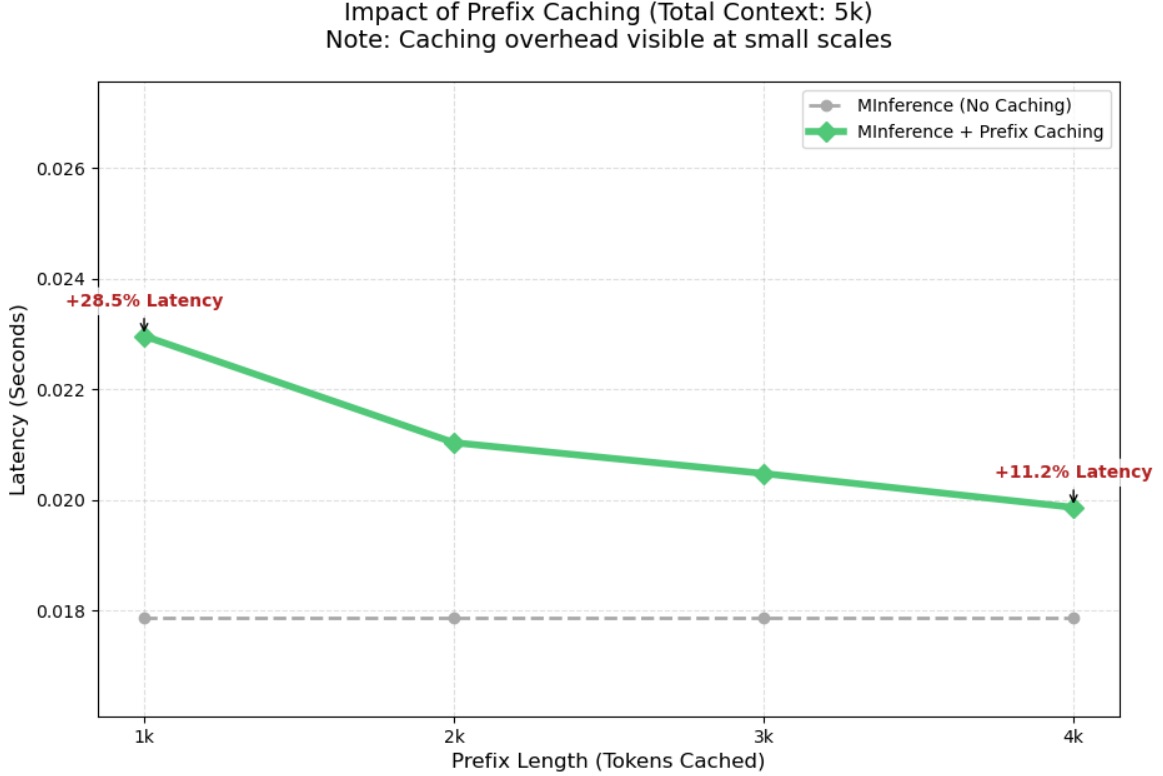


Figure 3: Attention kernel latency (seconds) for 50K context lengths across varying prefix cache fractions, compared with baseline. there is increase in latency because of paged attention overhead.

Remark on benchmark scope and practical speedup. The reported latency reductions represent a *lower bound* on the practical end-to-end speedup achievable by the proposed system. Because the benchmark isolates the attention kernel only, the feed-forward (MLP) layers are excluded from measurement. In a standard inference pass without prefix caching, the MLP must process every token in the sequence, contributing a cost that grows linearly with context length. Under vLLM’s prefix caching, however, the MLP computation for cached prefix tokens is skipped entirely, as their hidden states are already materialised. Consequently, in a full end-to-end setting, the effective speedup over the MInference baseline would be larger than the attention-only figures reported here, with the margin widening as the prefix fraction grows.

6.3 Output Fidelity

To verify that the vLLM integration does not degrade the numerical quality of the attention output relative to MInference, the output tensors of both systems are compared directly across all experimental configurations. The two implementations produce outputs that are numerically near-identical: the maximum elementwise absolute difference across all tested configurations is approximately 2×10^{-3} in bf16 precision.

This small discrepancy is expected and does not constitute an error in the proposed system. It arises from the non-associativity of floating-point arithmetic under parallel GPU execution: the order in which partial products are accumulated during block-sparse matrix multiplication is not guaranteed to be identical across implementations, and bf16’s reduced mantissa precision (7 bits, versus 23 bits for fp32) amplifies rounding differences between accumulation orderings [?]. Notably, these floating-point variances can lead the system to select different, yet semantically similar, blocks during top-k selection when multiple blocks possess nearly identical attention scores. Consequently, the observed difference ($\sim 10^{-3}$) is consistent with expected bf16 rounding error accumulation and the resulting stochasticity in block-sparse indexing. A formal end-to-end accuracy evaluation on long-context benchmarks (e.g., RULER,

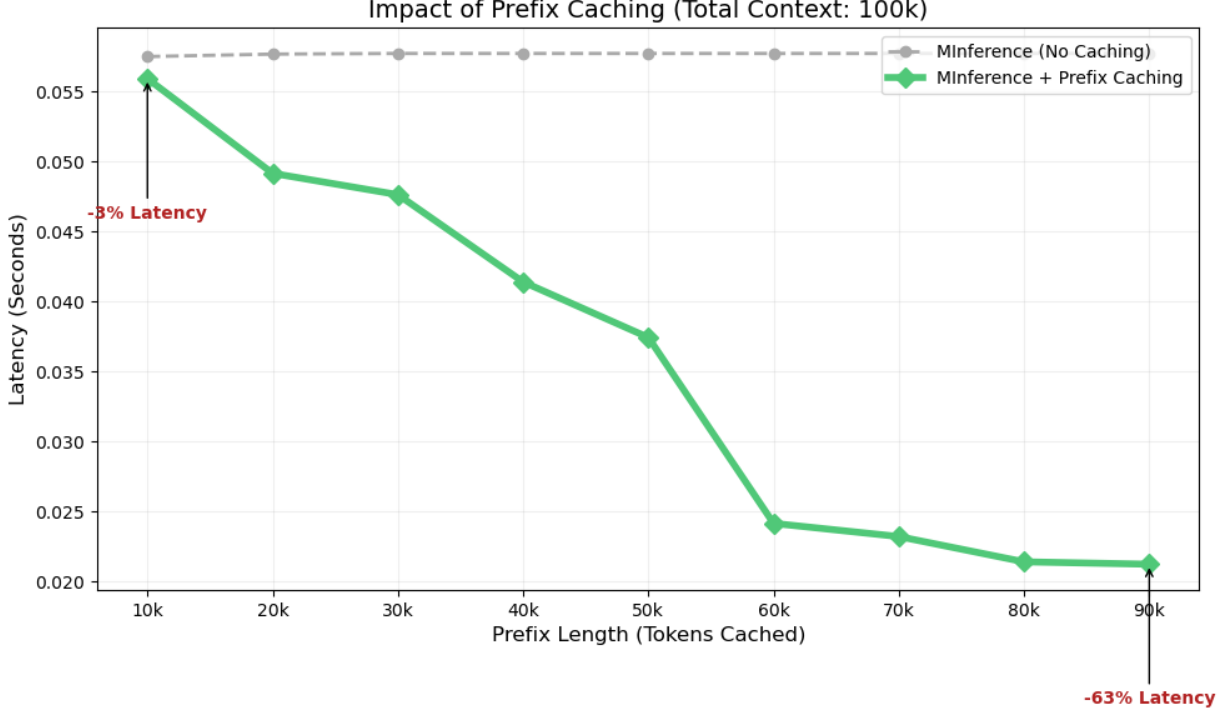


Figure 4: Attention kernel latency (seconds) for 100K context lengths across varying prefix cache fractions. There are significant reduction of latency, even shown in first 10k prefix

InfiniteBench) is left as future work, with the expectation that results will closely mirror those reported by MInference [?] given the near-identical attention outputs.

7 Theoretical Analysis

7.1 Latency Comparison: Traditional MInference vs. KV-Cache-Aware Block-Sparse Attention

We compare the pre-filling latency of the two systems described in Section ??: traditional MInference, which recomputes the full context on every request, and the proposed system, which operates over new-token projections only and reads the prefix KV from a paged cache.

Notation. Let n_p be the prefix length and n_q the new query length, so the full context has length $n = n_p + n_q$. Let π denote GPU FLOP throughput, M on-chip SRAM size, s the block selection ratio ($s \leq 0.05$ typically), and b the attention block size.

Traditional Block-Sparse Attention (MInference)

MInference receives the entire context as a single buffer $Q, K, V \in \mathbb{R}^{n \times d_h}$ with $n = n_p + n_q$, and processes the whole sequence in one pass. Two costs are incurred:

1. **Importance prediction.** The coarse score matrix $\hat{A} = \bar{Q}\bar{K}^\top$ requires a matrix multiply between the pooled query $\bar{Q} \in \mathbb{R}^{\lceil n/b \rceil \times d_h}$ and pooled key $\bar{K} \in \mathbb{R}^{\lceil n/b \rceil \times d_h}$, since the entire query sequence traverses the entire KV sequence:

$$t_{\text{predict}}^{\text{MInf}} \propto \frac{n}{b} \cdot \frac{n}{b} \cdot d_h = \frac{(n_p + n_q)^2 d_h}{b^2} \quad (17)$$

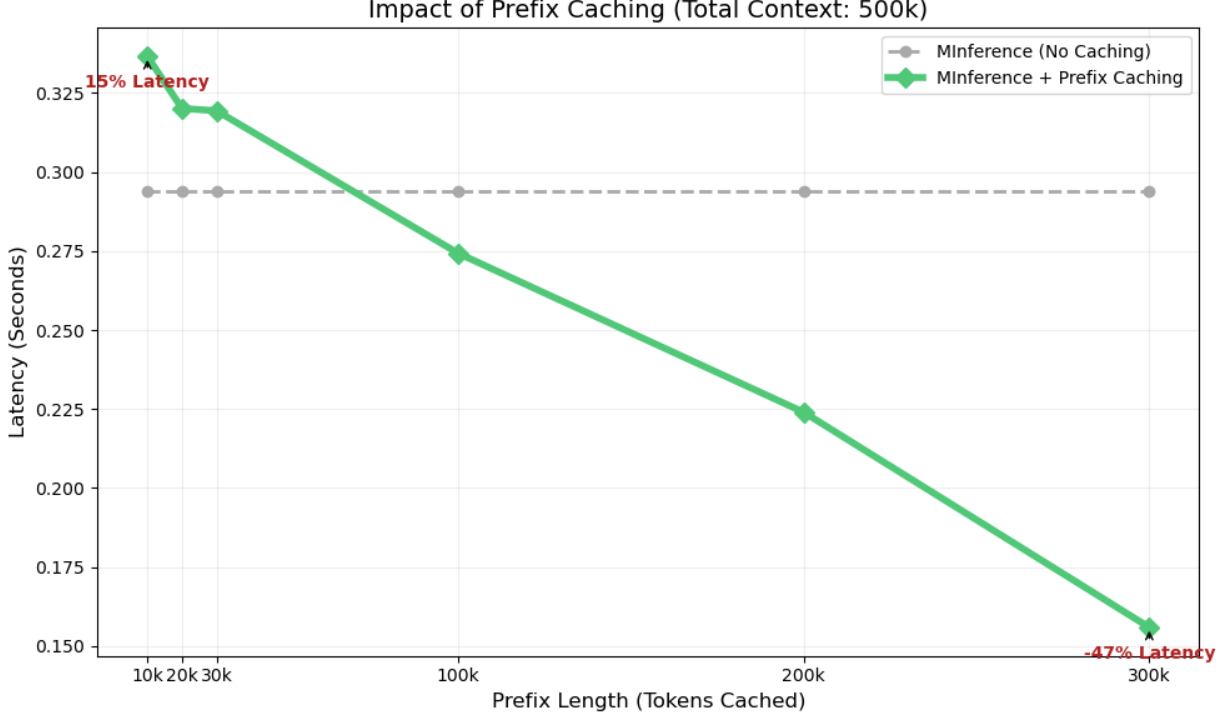


Figure 5: Attention kernel latency (seconds) for 500K context lengths across varying prefix cache fractions.

2. **Block-sparse attention.** The kernel executes over the selected fraction s of the square grid $\lceil n/b \rceil \times \lceil n/b \rceil$. Using the FlashAttention IO complexity $\Theta(n^2 d_h^2 / M)$ [?]:

$$t_{\text{attn}}^{\text{MInf}} = s \cdot \frac{n^2 d_h^2}{M \cdot \pi} = s \cdot \frac{(n_p + n_q)^2 d_h^2}{M \cdot \pi} \quad (18)$$

The total latency for traditional MInference is therefore:

$$t_{\text{total}}^{\text{MInf}} = \frac{(n_p + n_q)^2 d_h}{b^2 \pi} + s \cdot \frac{(n_p + n_q)^2 d_h^2}{M \cdot \pi} \quad (19)$$

Block-Sparse Attention with Paged KV Cache (Proposed)

The proposed system receives only the new-token projections $Q_{\text{new}}, K_{\text{new}}, V_{\text{new}} \in \mathbb{R}^{n_q \times d_h}$ and reads the prefix KV directly from cached pages via the block table ϕ . Two costs are incurred:

1. **Importance prediction.** The coarse score matrix requires only the new-token queries to traverse the full combined KV: $\bar{Q}_{\text{new}} \in \mathbb{R}^{\lceil n_q/b \rceil \times d_h}$ against $\bar{K}_{\text{full}} \in \mathbb{R}^{\lceil (n_p + n_q)/b \rceil \times d_h}$:

$$t_{\text{predict}}^{\text{proposed}} \propto \frac{n_q}{b} \cdot \frac{n_p + n_q}{b} \cdot d_h = \frac{n_q (n_p + n_q) d_h}{b^2} \quad (20)$$

2. **Block-sparse attention.** The kernel executes over the selected fraction s of the rectangular grid $\lceil n_q/b \rceil \times \lceil (n_p + n_q)/b \rceil$:

$$t_{\text{attn}}^{\text{proposed}} = s \cdot \frac{n_q (n_p + n_q) d_h^2}{M \cdot \pi} \quad (21)$$

The total latency for the proposed system is therefore:

$$t_{\text{total}}^{\text{proposed}} = \frac{n_q (n_p + n_q) d_h}{b^2 \pi} + s \cdot \frac{n_q (n_p + n_q) d_h^2}{M \cdot \pi} \quad (22)$$

Speedup Analysis

Both systems share the same structural form—prediction cost plus attention cost—but their scaling differs by a factor of $n_q/(n_p + n_q)$ in every term. The latency ratio is:

$$\frac{t_{\text{total}}^{\text{proposed}}}{t_{\text{total}}^{\text{MInf}}} = \frac{n_q (n_p + n_q) \left(\frac{d_h}{b^2} + \frac{s d_h^2}{M} \right)}{(n_p + n_q)^2 \left(\frac{d_h}{b^2} + \frac{s d_h^2}{M} \right)} = \frac{n_q}{n_p + n_q} \quad (23)$$

The ratio simplifies cleanly to $n_q/(n_p + n_q)$: the fraction of the full sequence that consists of *new* tokens. As the prefix fraction grows, $n_q/(n_p + n_q) \rightarrow 0$, and the proposed system’s latency vanishes relative to MInference. At the headline experimental configuration ($n_q = 10\text{K}$, $n_p = 90\text{K}$), the theoretical speedup is $10/(10 + 90) = 10\times$, confirming that the empirically observed 63% reduction is a conservative lower bound attributable to constant overheads not captured in the asymptotic model.

Thus the proposed system benefits doubly as the prefix fraction grows: it avoids prefix recomputation entirely, and its attention cost shrinks because only new-token queries drive the computation.

7.2 Approximation Error Bound

Let $O^* \in \mathbb{R}^{n_q \times d_h}$ be the true attention output for the new-token queries and $\hat{O} \in \mathbb{R}^{n_q \times d_h}$ the sparse approximation computed over $\hat{\mathcal{I}}$ only. For a block (r, c) , let A_{rc} denote the aggregated softmax weight and $V_c \in \mathbb{R}^{b \times d_h}$ the corresponding value block. The ℓ_∞ error is bounded by:

$$\|O^* - \hat{O}\|_\infty \leq \sum_{(r,c) \notin \hat{\mathcal{I}}} A_{rc} \cdot \|V_c\|_\infty \quad (24)$$

where $A_{rc} = \sum_{i \in \text{row } r, j \in \text{col } c} \text{softmax}(Q_i K_j^\top / \sqrt{d_h})$. The bound holds identically for both systems, since they compute equivalent attention outputs—the proposed system simply avoids recomputing the prefix KV that populates the column blocks. Jiang et al. [?] show that at $n = 128\text{K}$, $k_b = 4096$ captures 96.8% of total attention mass, leaving only 3.2% in the omitted tail. Whether this residual shrinks as n grows with fixed k_b remains an open question left for future work.

8 Discussion

8.1 The Isomorphism as a Design Principle

The PagedAttention–block-sparsity isomorphism identified in Section ?? is not merely an implementation convenience—it represents a **design principle** with broader implications. The block size b in sparse attention and the page size B in PagedAttention are the same structural unit: a contiguous range of B token positions whose KV vectors are stored and accessed together. Any inference system that manages KV caches in fixed-size blocks can directly leverage block-level importance scores to guide selective computation. This principle generalises naturally to multi-level memory hierarchies and future memory management designs.

8.2 Relationship to Prior Work

PagedAttention [?] introduces paged memory management for KV caches and prefix caching, but assumes dense attention access over all pages. FlashAttention [?, ?] achieves IO-efficient attention within GPU memory but does not address sparsity or paged layouts. MInference [?] reduces attention FLOPs via dynamic block sparsity but requires equal-length QKV sequences, preventing its use with any KV cache. This work is the first to bridge all three: block-sparse attention operating natively over a paged KV cache with prefix caching support, enabled by the structural isomorphism between attention block column indices and physical page addresses.

8.3 Limitations and Future Work

The primary limitation of this approach is the overhead of online importance prediction, which grows as $O((n/b)^2 d_h)$ and may become non-trivial for extremely long contexts ($n > 1\text{M}$). Future work may explore hierarchical prediction schemes or learned static masks per head to reduce this overhead. Additionally, the approximation error analysis assumes that mean pooling faithfully captures block importance; heads with highly peaked, non-smooth attention

distributions may require refined prediction strategies. A formal end-to-end evaluation on long-context benchmarks is also identified as important future work.

9 Conclusion

This work addresses a fundamental incompatibility between dynamic block-sparse attention and production LLM inference: MInference’s block-sparse kernel requires equal-length query, key, and value sequences, making it incompatible with KV caches and prefix caching. We identify and formalise the structural isomorphism that resolves this incompatibility: the column index c of an attention block directly corresponds to the physical KV page $\phi(c)$ in PagedAttention’s block table. Because block-sparse attention accesses KV vectors by block column index, and PagedAttention provides $O(1)$ lookup of physical pages by the same index, the two systems are naturally compatible without any data reorganisation, gather operations, or auxiliary structures.

By integrating MInference’s block-sparse importance predictor into vLLM, we enable block-sparse attention over prefix-cached KV pages. The system selectively attends only to the top- k_b most important pages per query row—skipping all others—while prefix caching eliminates redundant recomputation of shared context. Empirical evaluation demonstrates up to a 63% reduction in attention kernel latency at 100K tokens with a 90K token prefix cache, with near-identical output fidelity relative to the MInference baseline. The PagedAttention–block-sparsity isomorphism represents a fundamental design principle: any block-structured memory management system can directly leverage block-level importance scores to guide selective attention computation.

References

- [1] Huiqiang Jiang, Yucheng Li, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir H Abdi, Dongsheng Li, Chin-Yew Lin, et al. Minference 1.0: Accelerating pre-filling for long-context llms via dynamic sparse attention. *Advances in Neural Information Processing Systems*, 37:52481–52515, 2024.
- [2] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626, 2023.
- [3] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.
- [4] Gemini Team, Petko Georgiev, Ving Ian Lei, Ryan Burnell, Libin Bai, Anmol Gulati, Garrett Tanzer, Damien Vincent, Zhufeng Pan, Shibo Wang, Soroosh Mariooryad, Yifan Ding, Xinyang Geng, Fred Alcober, Roy Frostig, Mark Omernick, Lexi Walker, Cosmin Paduraru, Christina Sorokin, Andrea Tacchetti, Colin Gaffney, Samira Daruki, Olcan Sercinoglu, Zach Gleicher, Juliette Love, Paul Voigtlaender, Rohan Jain, Gabriela Surita, Kareem Mohamed, Rory Blevins, Junwhan Ahn, Tao Zhu, Kornraphop Kawintiranon, Orhan Firat, Yiming Gu, Yujing Zhang, Matthew Rahtz, Manaal Faruqui, Natalie Clay, Justin Gilmer, JD Co-Reyes, Ivo Penchev, Rui Zhu, Nobuyuki Morioka, Kevin Hui, Krishna Haridasan, Victor Campos, Mahdis Mahdih, Mandy Guo, Samer Hassan, Kevin Kilgour, Arpi Vezer, Heng-Tze Cheng, Raoul de Liedekerke, Siddharth Goyal, Paul Barham, DJ Strouse, Seb Noury, Jonas Adler, Mukund Sundararajan, Sharad Vikram, Dmitry Lepikhin, Michela Paganini, Xavier Garcia, Fan Yang, Dasha Valter, Maja Trebacz, Kiran Vodrahalli, Chulayuth Asawaroengchai, Roman Ring, Norbert Kalb, Livio Baldini Soares, Siddhartha Brahma, David Steiner, Tianhe Yu, Fabian Mentzer, Antoine He, Lucas Gonzalez, Bibo Xu, Raphael Lopez Kaufman, Laurent El Shafey, Junhyuk Oh, Tom Hennigan, George van den Driessche, Seth Odoom, Mario Lucic, Becca Roelofs, Sid Lall, Amit Marathe, Betty Chan, Santiago Ontanon, Luheng He, Denis Teplyashin, Jonathan Lai, Phil Crone, Bogdan Damoc, Lewis Ho, Sebastian Riedel, Karel Lenc, Chih-Kuan Yeh, Aakanksha Chowdhery, Yang Xu, Mehran Kazemi, Ehsan Amid, Anastasia Petrushkina, Kevin Swersky, Ali Khodaei, Gwoon Chen, Chris Larkin, Mario Pinto, Geng Yan, Adria Puigdomenech Badia, Piyush Patil, Steven Hansen, Dave Orr, Sebastien M. R. Arnold, Jordan Grimstad, Andrew Dai, Sholto Douglas, Rishika Sinha, Vikas Yadav, Xi Chen, Elena Gribovskaya, Jacob Austin, Jeffrey Zhao, Kaushal Patel, Paul Komarek, Sophia Austin, Sebastian Borgeaud, Linda Friso, Abhimanyu Goyal, Ben Caine, Kris Cao, Da-Woon Chung, Matthew Lamm, Gabe Barth-Maron, Thais Kagohara, Kate Olszewska, Mia Chen, Kaushik Shivakumar, Rishabh Agarwal, Harshal Godhia, Ravi Rajwar, Javier Snider, Xerxes Dotiwalla, Yuan Liu, Aditya Barua, Victor Ungureanu, Yuan Zhang, Bat-Or Gil Batsaikhan, Mateo Wirth, James Qin, Ivo Danihelka, Tulsee Doshi, Martin Chadwick, Jilin Chen, Sanil Jain, Quoc Le, Arjun Kar, Madhu Gurumurthy, Cheng Li, Ruoxin Sang, Fangyu Liu, Lampros Lamprou, Rich Munoz, Nathan Lintz, Harsh Mehta, Heidi Howard, Malcolm Reynolds, Lora Aroyo, Quan Wang, Lorenzo Blanco, Albin Cassirer, Jordan Griffith, Dipanjan Das, Stephan Lee, Jakub Sygnowski, Zach Fisher, James Besley, Richard Powell, Zafarali Ahmed, Dominik Paulus, David Reitter, Zalan Borsos, Rishabh Joshi, Aedan Pope, Steven Hand,

Vittorio Selo, Vihan Jain, Nikhil Sethi, Megha Goel, Takaki Makino, Rhys May, Zhen Yang, Johan Schalkwyk, Christina Butterfield, Anja Hauth, Alex Goldin, Will Hawkins, Evan Senter, Sergey Brin, Oliver Woodman, Marvin Ritter, Eric Noland, Minh Giang, Vijay Bolina, Lisa Lee, Tim Blyth, Ian Mackinnon, Machel Reid, Obaid Sarvana, David Silver, Alexander Chen, Lily Wang, Loren Maggiore, Oscar Chang, Nithya Attaluri, Gregory Thornton, Chung-Cheng Chiu, Oskar Bunyan, Nir Levine, Timothy Chung, Evgenii Eltyshov, Xiance Si, Timothy Lillicrap, Demetra Brady, Vaibhav Aggarwal, Boxi Wu, Yuanzhong Xu, Ross McIlroy, Kartikeya Badola, Paramjit Sandhu, Erica Moreira, Wojciech Stokowiec, Ross Hemsley, Dong Li, Alex Tudor, Pranav Shyam, Elahe Rahimtoroghi, Salem Haykal, Pablo Sprechmann, Xiang Zhou, Diana Mincu, Yujia Li, Ravi Addanki, Kalpesh Krishna, Xiao Wu, Alexandre Frechette, Matan Eyal, Allan Dafoe, Dave Lacey, Jay Whang, Thi Avrahami, Ye Zhang, Emanuel Taropa, Hanzhao Lin, Daniel Toyama, Eliza Rutherford, Motoki Sano, HyunJeong Choe, Alex Tomala, Chalence Safranek-Shrader, Nora Kassner, Mantas Pajarskas, Matt Harvey, Sean Sechrist, Meire Fortunato, Christina Lyu, Gamaleldin Elsayed, Chenkai Kuang, James Lottes, Eric Chu, Chao Jia, Chih-Wei Chen, Peter Humphreys, Kate Baumli, Connie Tao, Rajkumar Samuel, Cicero Nogueira dos Santos, Anders Andreassen, Nemanja Rakićević, Dominik Grewe, Aviral Kumar, Stephanie Winkler, Jonathan Caton, Andrew Brock, Sid Dalmia, Hannah Sheahan, Iain Barr, Yingjie Miao, Paul Natsev, Jacob Devlin, Feryal Behbahani, Flavien Prost, Yanhua Sun, Artiom Myaskovsky, Thanumalayan Sankaranarayanan Pillai, Dan Hurt, Angeliki Lazaridou, Xi Xiong, Ce Zheng, Fabio Pardo, Xiaowei Li, Dan Horgan, Joe Stanton, Moran Ambar, Fei Xia, Alejandro Lince, Mingqiu Wang, Basil Mustafa, Albert Webson, Hyo Lee, Rohan Anil, Martin Wicke, Timothy Dozat, Abhishek Sinha, Enrique Piqueras, Elahe Dabir, Shyam Upadhyay, Anudhyan Boral, Lisa Anne Hendricks, Corey Fry, Josip Djolonga, Yi Su, Jake Walker, Jane Labanowski, Ronny Huang, Vedant Misra, Jeremy Chen, RJ Skerry-Ryan, Avi Singh, Shruti Rijhwani, Dian Yu, Alex Castro-Ros, Beer Changpinyo, Romina Datta, Sumit Bagri, Arnar Mar Hrafnkelsson, Marcello Maggioni, Daniel Zheng, Yury Sulsky, Shaobo Hou, Tom Le Paine, Antoine Yang, Jason Riesa, Dominika Rogozinska, Dror Marcus, Dalia El Badawy, Qiao Zhang, Luyu Wang, Helen Miller, Jeremy Greer, Lars Lowe Sjos, Azade Nova, Heiga Zen, Rahma Chaabouni, Mihaela Rosca, Jiepu Jiang, Charlie Chen, Ruibo Liu, Tara Sainath, Maxim Krikun, Alex Polozov, Jean-Baptiste Lespiau, Josh Newlan, Zeyncep Cankara, Soo Kwak, Yunhan Xu, Phil Chen, Andy Coenen, Clemens Meyer, Katerina Tsihlias, Ada Ma, Juraj Gottweis, Jinwei Xing, Chenjie Gu, Jin Miao, Christian Frank, Zeynep Cankara, Sanjay Ganapathy, Ishita Dasgupta, Steph Hughes-Fitt, Heng Chen, David Reid, Keran Rong, Hongmin Fan, Joost van Amersfoort, Vincent Zhuang, Aaron Cohen, Shixiang Shane Gu, Anhad Mohanane, Anastasija Ilic, Taylor Tobin, John Wieting, Anna Bortsova, Phoebe Thacker, Emma Wang, Emily Caveness, Justin Chiu, Eren Sezener, Alex Kaskasoli, Steven Baker, Katie Millican, Mohamed Elhawaty, Kostas Aisopos, Carl Lebsack, Nathan Byrd, Hanjun Dai, Wenhao Jia, Matthew Wiethoff, Elnaz Davoodi, Albert Weston, Lakshman Yagati, Arun Ahuja, Isabel Gao, Golan Pundak, Susan Zhang, Michael Azzam, Khe Chai Sim, Sergi Caelles, James Keeling, Abhanshu Sharma, Andy Swing, YaGuang Li, Chenxi Liu, Carrie Grimes Bostock, Yamini Bansal, Zachary Nado, Ankesh Anand, Josh Lipschultz, Abhijit Karmarkar, Lev Proleev, Abe Ittycheriah, Soheil Hassas Yeganeh, George Polovets, Aleksandra Faust, Jiao Sun, Alban Rustemi, Pen Li, Rakesh Shivanna, Jeremiah Liu, Chris Welty, Federico Lebron, Anirudh Baddepudi, Sebastian Krause, Emilio Parisotto, Radu Soricut, Zheng Xu, Dawn Bloxwich, Melvin Johnson, Behnam Neyshabur, Justin Mao-Jones, Renshen Wang, Vinay Ramasesh, Zaheer Abbas, Arthur Guez, Constant Segal, Duc Dung Nguyen, James Svensson, Le Hou, Sarah York, Kieran Milan, Sophie Bridgers, Wiktor Gworek, Marco Tagliasacchi, James Lee-Thorp, Michael Chang, Alexey Guseynov, Ale Jakse Hartman, Michael Kwong, Ruizhe Zhao, Sheleem Kashem, Elizabeth Cole, Antoine Miech, Richard Tanburn, Mary Phuong, Filip Pavetic, Sebastien Cevey, Ramona Comanescu, Richard Ives, Sherry Yang, Cosmo Du, Bo Li, Zizhao Zhang, Mariko Iinuma, Clara Huiyi Hu, Aurko Roy, Shaan Bijwadia, Zhenkai Zhu, Danilo Martins, Rachel Saputro, Anita Gergely, Steven Zheng, Dawei Jia, Ioannis Antonoglou, Adam Sadosky, Shane Gu, Yingying Bi, Alek Andreev, Sina Samangooei, Mina Khan, Tomas Kocisky, Angelos Filos, Chintu Kumar, Colton Bishop, Adams Yu, Sarah Hodgkinson, Sid Mittal, Premal Shah, Alexandre Moufaret, Yong Cheng, Adam Bloniarz, Jaehoon Lee, Pedram Pejman, Paul Michel, Stephen Spencer, Vladimir Feinberg, Xuehan Xiong, Nikolay Savinov, Charlotte Smith, Siamak Shakeri, Dustin Tran, Mary Chesus, Bernd Bohnet, George Tucker, Tamara von Glehn, Carrie Muir, Yiran Mao, Hideto Kazawa, Ambrose Slone, Kedar Soparkar, Disha Shrivastava, James Cobon-Kerr, Michael Sharman, Jay Pavagadhi, Carlos Araya, Karolis Misiunas, Nimesh Ghelani, Michael Laskin, David Barker, Qiujia Li, Anton Briukhov, Neil Houlsby, Mia Glaese, Balaji Lakshminarayanan, Nathan Schucher, Yunhao Tang, Eli Collins, Hyeontaek Lim, Fangxiaoyu Feng, Adria Recasens, Guangda Lai, Alberto Magni, Nicola De Cao, Aditya Siddhant, Zoe Ashwood, Jordi Orbay, Mostafa Dehghani, Jenny Brennan, Yifan He, Kelvin Xu, Yang Gao, Carl Saroufim, James Molloy, Xinyi Wu, Seb Arnold, Solomon Chang, Julian Schrittwieser, Elena Buchatskaya, Soroush Radpour, Martin Polacek, Skye Giordano, Ankur Bapna, Simon Tokumine, Vincent Hellendoorn, Thibault Sottiaux, Sarah Cogan, Aliaksei Severyn, Mohammad Saleh, Shantanu Thakoor, Laurent Shefey, Siyuan Qiao, Meenu Gaba, Shuo yiin Chang, Craig Swanson, Biao Zhang, Benjamin Lee, Paul Kishan Rubenstein, Gan Song, Tom Kwiatkowski, Anna Koop, Ajay Kannan, David Kao, Parker Schuh, Axel Stjerngren, Golnaz Ghiasi, Gena Gibson, Luke Vilnis, Ye Yuan, Felipe Tiengo Ferreira, Aishwarya Kamath, Ted Klimenko, Ken Franko, Kefan Xiao, Indro Bhattacharya, Miteyan Patel, Rui Wang, Alex Morris, Robin Strudel, Vivek Sharma, Peter Choy,

Sayed Hadi Hashemi, Jessica Landon, Mara Finkelstein, Priya Jhakra, Justin Frye, Megan Barnes, Matthew Mauger, Dennis Daun, Khuslen Baatarsukh, Matthew Tung, Wael Farhan, Henryk Michalewski, Fabio Viola, Felix de Chaumont Quirry, Charline Le Lan, Tom Hudson, Qingze Wang, Felix Fischer, Ivy Zheng, Elspeth White, Anca Dragan, Jean baptiste Alayrac, Eric Ni, Alexander Pritzel, Adam Iwanicki, Michael Isard, Anna Bulanova, Lukas Zilka, Ethan Dyer, Devendra Sachan, Srivatsan Srinivasan, Hannah Muckenhirn, Honglong Cai, Amol Mandhane, Mukarram Tariq, Jack W. Rae, Gary Wang, Kareem Ayoub, Nicholas FitzGerald, Yao Zhao, Woohyun Han, Chris Alberti, Dan Garrette, Kashyap Krishnakumar, Mai Gimenez, Anselm Levskaya, Daniel Sohn, Josip Matak, Inaki Iturrate, Michael B. Chang, Jackie Xiang, Yuan Cao, Nishant Ranka, Geoff Brown, Adrian Hutter, Vahab Mirrokni, Nanxin Chen, Kaisheng Yao, Zoltan Egyed, Francois Galilee, Tyler Liechty, Praveen Kallakuri, Evan Palmer, Sanjay Ghemawat, Jasmine Liu, David Tao, Chloe Thornton, Tim Green, Mimi Jasarevic, Sharon Lin, Victor Cotruta, Yi-Xuan Tan, Noah Fiedel, Hongkun Yu, Ed Chi, Alexander Neitz, Jens Heitkaemper, Anu Sinha, Denny Zhou, Yi Sun, Charbel Kaed, Brice Hulse, Swaroop Mishra, Maria Georgaki, Sneha Kudugunta, Clement Farabet, Izhak Shafran, Daniel Vlasic, Anton Tsitsulin, Rajagopal Ananthanarayanan, Alen Carin, Guolong Su, Pei Sun, Shashank V, Gabriel Carvajal, Josef Broder, Iulia Comsa, Alena Repina, William Wong, Warren Weilun Chen, Peter Hawkins, Egor Filonov, Lucia Loher, Christoph Hirnschall, Weiye Wang, Jingchen Ye, Andrea Burns, Hardie Cate, Diana Gage Wright, Federico Piccinini, Lei Zhang, Chu-Cheng Lin, Ionel Gog, Yana Kulizhskaya, Ashwin Sreevatsa, Shuang Song, Luis C. Cobo, Anand Iyer, Chetan Tekur, Guillermo Garrido, Zhu Yun Xiao, Rupert Kemp, Huaixiu Steven Zheng, Hui Li, Ananth Agarwal, Christel Ngani, Kati Goshvadi, Rebeca Santamaria-Fernandez, Wojciech Fica, Xinyun Chen, Chris Gorgolewski, Sean Sun, Roopal Garg, Xinyu Ye, S. M. Ali Eslami, Nan Hua, Jon Simon, Pratik Joshi, Yelin Kim, Ian Tenney, Sahitya Potluri, Lam Nguyen Thiet, Quan Yuan, Florian Luisier, Alexandra Chronopoulou, Salvatore Scellato, Praveen Srinivasan, Minmin Chen, Vinod Koverkathu, Valentin Dalibard, Yaming Xu, Brennan Saeta, Keith Anderson, Thibault Sellam, Nick Fernando, Fantine Huot, Junehyuk Jung, Mani Varadarajan, Michael Quinn, Amit Raul, Maigo Le, Ruslan Habalov, Jon Clark, Komal Jalan, Kalesha Bullard, Achintya Singhal, Thang Luong, Boyu Wang, Sujevan Rajayogam, Julian Eisenschlos, Johnson Jia, Daniel Finchelstein, Alex Yakubovich, Daniel Balle, Michael Fink, Sameer Agarwal, Jing Li, Dj Dvijotham, Shalini Pal, Kai Kang, Jaclyn Konzelmann, Jennifer Beattie, Olivier Dousse, Diane Wu, Remi Crocker, Chen Elkind, Siddhartha Reddy Jonnalagadda, Jong Lee, Dan Holtmann-Rice, Krystal Kallarackal, Rosanne Liu, Denis Vnukov, Neera Vats, Luca Invernizzi, Mohsen Jafari, Huanjie Zhou, Lilly Taylor, Jennifer Prendki, Marcus Wu, Tom Eccles, Tianqi Liu, Kavya Kopparapu, Francoise Beaufays, Christof Angermueller, Andreea Marzoca, Shourya Sarcar, Hilal Dib, Jeff Stanway, Frank Perbet, Nejc Trdin, Rachel Sterneck, Andrey Khorlin, Dinghua Li, Xihui Wu, Sonam Goenka, David Madras, Sasha Goldshtein, Willi Gierke, Tong Zhou, Yaxin Liu, Yannie Liang, Anais White, Yunjie Li, Shreya Singh, Sanaz Bahargam, Mark Epstein, Sujoy Basu, Li Lao, Adnan Ozturk, Carl Crous, Alex Zhai, Han Lu, Zora Tung, Neeraj Gaur, Alanna Walton, Lucas Dixon, Ming Zhang, Amir Globerson, Grant Uy, Andrew Bolt, Olivia Wiles, Milad Nasr, Ilia Shumailov, Marco Selvi, Francesco Piccinno, Ricardo Aguilar, Sara McCarthy, Misha Khalman, Mrinal Shukla, Vlado Galic, John Carpenter, Kevin Vilella, Haibin Zhang, Harry Richardson, James Martens, Matko Bosnjak, Shreyas Rammohan Belle, Jeff Seibert, Mahmoud Alnahlawi, Brian McWilliams, Sankalp Singh, Annie Louis, Wen Ding, Dan Popovici, Lenin Simicich, Laura Knight, Pulkit Mehta, Nishesh Gupta, Chongyang Shi, Saaber Fatehi, Jovana Mitrovic, Alex Grills, Joseph Pagadora, Tsendsuren Munkhdalai, Dessie Petrova, Danielle Eisenbud, Zhishuai Zhang, Damion Yates, Bhavishya Mittal, Nilesch Tripuraneni, Yannis Assael, Thomas Brovelli, Prateek Jain, Mihajlo Velimirovic, Canfer Akbulut, Jiaqi Mu, Wolfgang Macherey, Ravin Kumar, Jun Xu, Haroon Qureshi, Gheorghe Comanici, Jeremy Wiesner, Zhitao Gong, Anton Ruddock, Matthias Bauer, Nick Felt, Anirudh GP, Anurag Arnab, Dustin Zelle, Jonas Rothfuss, Bill Rosgen, Ashish Shenoy, Bryan Seybold, Xinjian Li, Jayaram Mudigonda, Goker Erdogan, Jiawei Xia, Jiri Simsa, Andrea Michi, Yi Yao, Christopher Yew, Steven Kan, Isaac Caswell, Carey Radebaugh, Andre Elisseeff, Pedro Valenzuela, Kay McKinney, Kim Paterson, Albert Cui, Eri Latorre-Chimoto, Solomon Kim, William Zeng, Ken Durden, Priya Ponnappalli, Tiberiu Sosea, Christopher A. Choquette-Choo, James Manyika, Brona Robenek, Harsha Vashisht, Sebastien Pereira, Hoi Lam, Marko Velic, Denese Owusu-Afriyie, Katherine Lee, Tolga Bolukbasi, Alicia Parrish, Shawn Lu, Jane Park, Balaji Venkatraman, Alice Talbert, Lambert Rosique, Yuchung Cheng, Andrei Sozanschi, Adam Paszke, Praveen Kumar, Jessica Austin, Lu Li, Khalid Salama, Bartek Perz, Wooyeol Kim, Nandita Dukkhipati, Anthony Baryshnikov, Christos Kaplanis, XiangHai Sheng, Yuri Chervonyi, Caglar Unlu, Diego de Las Casas, Harry Askham, Kathryn Tunyasuvunakool, Felix Gimeno, Siim Poder, Chester Kwak, Matt Miecnikowski, Vahab Mirrokni, Alek Dimitriev, Aaron Parisi, Danyu Liu, Tomy Tsai, Toby Shelvane, Christina Kouridi, Drew Garmon, Adrian Goedeckemeyer, Adam R. Brown, Anitha Vijayakumar, Ali Elqursh, Sadegh Jazayeri, Jin Huang, Sara Mc Carthy, Jay Hoover, Lucy Kim, Sandeep Kumar, Wei Chen, Courtney Biles, Garrett Bingham, Evan Rosen, Lisa Wang, Qijun Tan, David Engel, Francesco Pongetti, Dario de Cesare, Dongseong Hwang, Lily Yu, Jennifer Pullman, Srini Narayanan, Kyle Levin, Siddharth Gopal, Megan Li, Asaf Aharoni, Trieu Trinh, Jessica Lo, Norman Casagrande, Roopali Vij, Loic Matthey, Bramandia Ramadhana, Austin Matthews, CJ Carey, Matthew Johnson, Kremena Goranova, Rohin Shah, Shereen Ashraf, Kingshuk Dasgupta, Rasmus Larsen, Yicheng Wang, Manish Reddy Vuyyuru, Chong Jiang, Joana Ijazi, Kazuki Osawa, Celine Smith, Ramya Sree Boppana, Taylan Bilal, Yuma Koizumi, Ying Xu, Yasemin

- Altun, Nir Shabat, Ben Bariach, Alex Korchemniy, Kiam Choo, Olaf Ronneberger, Chimezie Iwuanyanwu, Shubin Zhao, David Soergel, Cho-Jui Hsieh, Irene Cai, Shariq Iqbal, Martin Sundermeyer, Zhe Chen, Elie Bursztein, Chaitanya Malaviya, Fadi Biadisy, Prakash Shroff, Inderjit Dhillon, Tejasi Latkar, Chris Dyer, Hannah Forbes, Massimo Nicosia, Vitaly Nikolaev, Somer Greene, Marin Georgiev, Pidong Wang, Nina Martin, Hanie Sedghi, John Zhang, Praseem Banzal, Doug Fritz, Vikram Rao, Xuezhi Wang, Jiageng Zhang, Viorica Patraucean, Dayou Du, Igor Mordatch, Ivan Jurin, Lewis Liu, Ayush Dubey, Abhi Mohan, Janek Nowakowski, Vlad-Doru Ion, Nan Wei, Reiko Tojo, Maria Abi Raad, Drew A. Hudson, Vaishakh Keshava, Shubham Agrawal, Kevin Ramirez, Zhichun Wu, Hoang Nguyen, Ji Liu, Madhavi Sewak, Bryce Petrini, DongHyun Choi, Ivan Philips, Ziyue Wang, Ioana Bica, Ankush Garg, Jarek Wilkiewicz, Priyanka Agrawal, Xiaowei Li, Danhao Guo, Emily Xue, Naseer Shaik, Andrew Leach, Sadh MNM Khan, Julia Wiesinger, Sammy Jerome, Abhishek Chakladar, Alek Wenjiao Wang, Tina Ornduff, Folake Abu, Alireza Ghaffarkhah, Marcus Wainwright, Mario Cortes, Frederick Liu, Joshua Maynez, Andreas Terzis, Pouya Samangouei, Riham Mansour, Tomasz Kępa, François-Xavier Aubet, Anton Algymer, Dan Banica, Agoston Weisz, Andras Orban, Alexandre Senges, Ewa Andrejczuk, Mark Geller, Niccolo Dal Santo, Valentin Anklin, Majd Al Merey, Martin Baeuml, Trevor Strohman, Junwen Bai, Slav Petrov, Yonghui Wu, Demis Hassabis, Koray Kavukcuoglu, Jeff Dean, and Oriol Vinyals. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context, 2024.
- [5] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 37:62557–62583, 2024.
- [6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023.
- [7] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems*, 35:16344–16359, 2022.
- [8] Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax, 2018.
- [9] Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. Society for Industrial and Applied Mathematics, 2nd edition, 2002.